



Universidad Nacional Experimental de Guayana.
Vicerrectorado Académico.
Coordinación General de Pregrado.
Proyecto de Carrera: Ingeniería en Informática.
Unidad Curricular: Lenguajes y Compiladores.
Semestre 7 – Sección: 1 y 2.

Análisis Léxico.

Los ASTronautas

“Explorando el Universo de los Lenguajes, un Nodo a la vez”

Profesor:

Ing. Félix Márquez

Estudiantes:

C.I.: V-25.933.680. Alburquerque Sheen

C.I.: V-30.907.427. Antoima Mariangel

C.I.: V-28.475.271. García Carlos

C.I.: V-24.848.424. Varguillas Génesis.

Ciudad Guayana, Julio del 2026.

Índice.

Contenido	pág
Introducción	3
4. Análisis Léxico.....	4
4.1. Autómatas Finitos y Expresiones Regulares	4
4.1.1. Autómatas Finitos.....	4
4.1.2 Expresiones Regulares.	5
4.1.3. Equivalencia entre autómatas finitos y expresiones regulares.	5
4.1.4. Aplicación en el análisis léxico.....	5
4.2. Construcción de autómatas finitos a partir de gramáticas regulares.....	5
4.2.1. Fundamento de la construcción: de la gramática al autómata.....	6
4.2.2. De la gramática regular al autómata determinista (AFD).....	7
4.3. Autómatas de Pila.....	8
4.3.1. Relación con otros autómatas y jerarquía de Chomsky	9
4.3.2. Relación con AFD y AFND	10
4.4. Construcción de analizadores léxicos desde el autómata.	10
4.4.1. Enfoques para la implementación de analizadores léxicos:.....	10
4.4.2. El proceso de tokenización	11
4.5. Construcción de analizadores léxicos mediante metacompiladores.	11
4.5.1. Metacompiladores y análisis léxico.....	12
4.5.2. Ventajas del uso de metacompiladores	12
Actividad 1: Defina y presente ejemplos de autómata de pilas.	13
Actividad 2: Lexer para archivos docker mediante expresiones regulares	20
Actividad 3: Lexer para L utilizando metacompilador.	24
Actividad 4: FLEX	27
Conclusión.	37
Referencias.....	39

Introducción

El análisis léxico constituye la primera fase del proceso de compilación y, aunque a simple vista pueda parecer la más sencilla, es fundamental para el éxito de todo el proceso de traducción de un lenguaje de programación. Esta etapa se encarga de transformar el flujo de caracteres que componen el código fuente en una secuencia estructurada de tokens, que son las unidades mínimas con significado dentro de un lenguaje formal. Sin un análisis léxico correcto, las fases posteriores del compilador (como el análisis sintáctico o el semántico) no podrían funcionar adecuadamente, ya que carecerían de una representación clara y organizada del programa fuente.

En los temas anteriores de la asignatura, se abordaron los fundamentos teóricos de los lenguajes formales, las gramáticas y la Jerarquía de Chomsky, estableciendo las bases para comprender cómo se generan y reconocen los lenguajes. El presente tema, denominado Análisis Léxico, materializa esos conceptos al llevarlos a la práctica mediante la construcción de analizadores léxicos, tanto desde un enfoque manual basado en expresiones regulares como utilizando herramientas de metacompilación como Flex.

Este informe se estructura en dos grandes bloques. En primer lugar, se presenta una investigación teórica que abarca los autómatas finitos y su relación con las expresiones regulares, la construcción de autómatas a partir de gramáticas regulares, los autómatas de pila como modelos de computación con mayor poder expresivo, y las metodologías para construir analizadores léxicos desde el autómata o mediante metacompiladores. Este marco conceptual permite comprender los fundamentos matemáticos que sustentan el análisis léxico y su importancia en el contexto de los compiladores.

En segundo lugar, se desarrollan cuatro actividades prácticas que permiten aplicar los conceptos teóricos estudiados. La primera actividad aborda la definición y ejemplificación de los autómatas de pila, comparando su poder computacional con el de los autómatas finitos. La segunda actividad consiste en la construcción de un analizador léxico en Python para verificar archivos Dockerfile mediante expresiones regulares. La tercera actividad implica el diseño de un lenguaje L, subconjunto de Rust, y la implementación de un lexer utilizando Flex como metacompilador. Finalmente, la cuarta actividad explora el uso de FLEX en el área de seguridad informática, identificando lenguajes relevantes y proponiendo su tokenización.

En conjunto, este trabajo busca consolidar los conocimientos teóricos del análisis léxico y demostrar su aplicación en problemas reales de procesamiento de lenguajes. La comprensión de estos conceptos es esencial para cualquier ingeniero en informática, ya que constituye la base para la construcción de compiladores, intérpretes y herramientas de procesamiento de lenguaje.

4. Análisis Léxico

El análisis léxico constituye la primera fase del proceso de compilación. Su función principal consiste en leer el flujo de caracteres que conforman el programa fuente y agruparlos en una secuencia de tokens, que son las unidades mínimas con significado dentro de un lenguaje de programación (Aho, Sethi y Ullman, 1998).

Un token es una cadena con un significado asignado y se estructura como un par que consta de un «nombre de token» y un «valor de token» opcional (Analizador léxico, s.f.). Los nombres de token comunes incluyen categorías como identificador (nombres elegidos por el programador), palabra clave (nombres ya presentes en el lenguaje, como `if` o `while`), operador (símbolos que operan sobre argumentos), literal (valores numéricos o textuales), y separador (signos de puntuación como `;` o `{}`) (Analizador léxico, s.f.).

Este proceso es fundamental porque transforma el código fuente, que para la máquina no es más que una cadena de caracteres, en una estructura comprensible que las fases posteriores del compilador podrá procesar adecuadamente. Generalmente, el analizador léxico omite los espacios en blanco, las tabulaciones y los comentarios, ya que estos no son relevantes para la sintaxis del programa, aunque sí lo son para la legibilidad del código (Compiladores1.webnode.mx, 2024). Cuando el analizador léxico encuentra un carácter que no puede clasificar dentro de ningún patrón definido para el lenguaje, se genera un error léxico, permitiendo al programador identificar y corregir problemas en las primeras etapas del desarrollo (Compiladores1.webnode.mx, 2024).

La etapa de análisis léxico está basada usualmente en una máquina de estados finitos, y la especificación de un lenguaje a menudo incluye un conjunto de reglas que definen el léxico, consistentes comúnmente en expresiones regulares (Analizador léxico, s.f.).

4.1. Autómatas Finitos y Expresiones Regulares

4.1.1. Autómatas Finitos.

Un autómata finito es un modelo matemático de computación que consta de un conjunto finito de estados, un alfabeto de entrada, una función de transición, un estado inicial y un conjunto de estados de aceptación (Hopcroft, Motwani y Ullman, 2002). Existen dos variantes principales:

- ✓ Autómata Finito Determinista (AFD): En cada estado y para cada símbolo de entrada, existe exactamente una transición posible hacia otro estado. El comportamiento del autómata está completamente determinado por la entrada.
- ✓ Autómata Finito No Determinista (AFND): En un mismo estado y para un mismo símbolo de entrada, puede haber múltiples transiciones posibles, incluyendo transiciones

épsilon (sin consumir símbolo). Esto permite varias rutas de ejecución para una misma entrada (Hopcroft, Motwani y Ullman, 2002).

4.1.2 Expresiones Regulares.

Una expresión regular es una notación algebraica utilizada para describir patrones de cadenas sobre un alfabeto determinado (Hopcroft, Motwani y Ullman, 2002). Se construye a partir de símbolos del alfabeto utilizando operadores como concatenación, unión (alternancia) y clausura de Kleene (repetición). En el contexto del análisis léxico, las expresiones regulares se emplean para definir los patrones que debe reconocer el analizador para identificar cada tipo de token (Aho, Sethi y Ullman, 1998). Por ejemplo, una expresión regular para reconocer un número entero podría ser $[0-9]^+$, que significa "uno o más dígitos".

4.1.3. Equivalencia entre autómatas finitos y expresiones regulares.

El Teorema de Kleene establece que existe una equivalencia fundamental entre los autómatas finitos y las expresiones regulares (Hopcroft, Motwani y Ullman, 2002). Esto significa que todo lenguaje que puede ser descrito por una expresión regular puede ser reconocido por un autómata finito, y viceversa. Esta relación es la base teórica de los analizadores léxicos.

4.1.4. Aplicación en el análisis léxico.

La conexión entre autómatas y expresiones regulares es lo que permite construir analizadores léxicos. El proceso sigue estos pasos:

- ✓ Se definen las expresiones regulares que describen los patrones de cada tipo de token (Aho, Sethi y Ullman, 1998).
- ✓ A partir de estas expresiones regulares, se construye un autómata finito que reconoce los tokens correspondientes (Hopcroft, Motwani y Ullman, 2002).
- ✓ El autómata se implementa como un programa que recorre carácter por carácter el código fuente, cambiando de estado según la entrada, hasta identificar los tokens o detectar errores léxicos (Aho, Sethi y Ullman, 1998).

Este enfoque permite a los diseñadores de lenguajes especificar la estructura léxica de manera concisa mediante expresiones regulares, mientras que la implementación se apoya en la teoría de autómatas para construir analizadores eficientes y correctos.

4.2. Construcción de autómatas finitos a partir de gramáticas regulares.

La estrecha relación entre las gramáticas regulares y los autómatas finitos constituye uno de los pilares fundamentales de la teoría de lenguajes formales. Esta conexión permite representar un mismo lenguaje regular mediante dos formalismos equivalentes: el generativo (gramáticas) y el reconocedor (autómatas) (Hopcroft, Motwani & Ullman, 2002).

Gramáticas regulares y su clasificación:

Una gramática regular es aquella que cumple con restricciones específicas en sus reglas de producción (Hopcroft et al., 2002). Formalmente, una gramática $G=(V,T,S,P)$ se clasifica como regular por la derecha (right-linear) si todas sus producciones son de la forma:

$$A \rightarrow xB \text{ o } A \rightarrow x$$

donde $A, B \in V$ (no terminales) y $x \in T^*$ (cadenas de terminales) (Ortega de la Puente et al., 2006). De manera análoga, una gramática es regular por la izquierda (left-linear) si sus producciones tienen la forma:

$$A \rightarrow Bx \text{ o } A \rightarrow x$$

Un aspecto crucial es que una gramática es considerada regular únicamente si todas sus producciones son consistentes con uno de estos dos tipos; una mezcla de reglas por la derecha y por la izquierda no define una gramática regular (Hopcroft et al., 2002).

4.2.1. Fundamento de la construcción: de la gramática al autómata

El principio que permite la construcción de un autómata a partir de una gramática regular es la correspondencia directa entre el proceso de derivación de la gramática y el funcionamiento de un autómata finito (Hopcroft et al., 2002). Durante una derivación en una gramática regular por la derecha, las formas sentenciales tienen una estructura particular: una secuencia de terminales seguida de un único no terminal. El autómata finito imita este proceso consumiendo los terminales generados en cada paso y realizando transiciones entre estados que representan los no terminales de la gramática (Ortega de la Puente et al., 2006).

Algoritmo de construcción (para gramáticas regulares por la derecha)

El teorema que establece la equivalencia entre gramáticas regulares y autómatas finitos demuestra que para toda gramática regular G , existe un autómata finito (generalmente no determinista, NFA) que reconoce exactamente el mismo lenguaje $L(A)=L(G)$ (Hopcroft et al., 2002).

Dada una gramática $G=(T,N,P,Z)$, donde Z es el símbolo inicial, se construye un autómata $A=(T,N \cup \{f\},R,Z,F)$ siguiendo los siguientes pasos (Ortega de la Puente et al., 2006):

- ✓ **Estados:** El conjunto de estados Q está formado por los no terminales de la gramática (N) más un nuevo estado final f ($f \notin N$). El estado inicial es el símbolo inicial Z .
- ✓ **Transiciones y reglas de producción:** Para cada producción en P , se genera una regla de transición en R :

- Si existe una producción de la forma $X \rightarrow t$, donde $X \in N$ y $t \in T$, se añade una transición $Xt \rightarrow f$. Esto significa que desde el estado X , al leer el terminal t , se pasa al estado final f .
 - Si existe una producción de la forma $X \rightarrow tY$, con $X, Y \in N$ y $t \in T$, se añade una transición $Xt \rightarrow Y$. Esto significa que desde el estado X , al leer el terminal t , se pasa al estado Y .
- ✓ **Estados de aceptación:** El conjunto de estados finales F está compuesto por el estado f y por todos aquellos no terminales X que tengan una producción a la cadena vacía ($X \rightarrow \epsilon \in P$) (Hopcroft et al., 2002).

Aplicación y ejemplo práctico:

Para ilustrar el proceso, considérese la siguiente gramática regular que genera el lenguaje $L = \{(ab)^*a\}$ (Linz, 2011):

- ✓ No terminales: $N = \{V0, V1\}$
- ✓ Terminales: $T = \{a, b\}$
- ✓ Símbolo inicial: $V0$
- ✓ Producciones P :
 - $V0 \rightarrow aV1$
 - $V1 \rightarrow abV0$
 - $V1 \rightarrow b$

Para construir el autómata, se inicia el grafo de transiciones con los estados $V0$, $V1$ y se añade el estado final f (Linz, 2011):

- ✓ La producción $V0 \rightarrow aV1$ crea una arista etiquetada con a entre $V0$ y $V1$.
- ✓ La producción $V1 \rightarrow abV0$ requiere una ruta entre $V1$ y $V0$ que consuma la cadena ab . Para esto, se introduce un estado intermedio, de modo que se tenga: $V1 \rightarrow aq \rightarrow bV0$ (Linz, 2011).
- ✓ La producción $V1 \rightarrow b$ crea una transición $V1 \rightarrow bf$.

El autómata resultante es un NFA que reconoce el lenguaje de la gramática original (Linz, 2011).

4.2.2. De la gramática regular al autómata determinista (AFD)

El procedimiento descrito generalmente produce un autómata finito no determinista (NFA) (Hopcroft et al., 2002). Para obtener un autómata finito determinista (AFD), se debe aplicar la construcción de subconjuntos (o powerset construction) al NFA resultante (Ortega de

la Puente et al., 2006). Sin embargo, existe la posibilidad de construir un AFD directamente a partir de la gramática, interpretando cada estado como un conjunto de no terminales de la gramática (Hopcroft et al., 2002).

4.3. Autómatas de Pila

Un autómata de pila (Pushdown Automaton, PDA) es un modelo matemático de computación que extiende la capacidad de los autómatas finitos mediante la incorporación de una memoria auxiliar en forma de pila (Hopcroft, Motwani & Ullman, 2002). Formalmente, un autómata de pila se define como una séxtupla:

$$M=(Q,\Sigma,\Gamma,\delta,q_0,Z_0,F)$$

donde (Hopcroft et al., 2002):

- ✓ Q es un conjunto finito de estados.
- ✓ Σ es un alfabeto finito de entrada.
- ✓ Γ es un alfabeto finito de símbolos de pila.
- ✓ $\delta:Q\times(\Sigma\cup\{\epsilon\})\times\Gamma\rightarrow P(Q\times\Gamma^*)$ es la función de transición (Ortega de la Puente et al., 2006).
- ✓ $q_0\in Q$ es el estado inicial.
- ✓ $Z_0\in\Gamma$ es el símbolo inicial de la pila.
- ✓ $F\subseteq Q$ es el conjunto de estados de aceptación.

El funcionamiento de un autómata de pila se basa en la lectura de la entrada carácter por carácter y en la manipulación de la pila mediante operaciones de apilamiento (push) y desapilamiento (pop) (Hopcroft et al., 2002). La función de transición determina, a partir del estado actual, el símbolo leído y el tope de la pila, el nuevo estado y la secuencia de símbolos que reemplazarán el tope de la pila (Ortega de la Puente et al., 2006).

Ejemplo práctico: Lenguaje $a^n b^n$

Un ejemplo clásico que ilustra la potencia de los autómatas de pila es el reconocimiento del lenguaje $\{a^n b^n | n \geq 0\}$, que consiste en cadenas con el mismo número de letras 'a' y 'b' (Hopcroft et al., 2002). Este lenguaje no puede ser reconocido por un autómata finito (determinista o no determinista), ya que su reconocimiento requiere contar el número de 'a' para verificar que coincide con el número de 'b' (Ortega de la Puente et al., 2006). Sin embargo, un autómata de pila sí puede reconocerlo mediante el siguiente mecanismo:

- ✓ Para cada símbolo 'a' leído, se apila un símbolo (por ejemplo, A).

- ✓ Cuando se comienzan a leer las 'b', se desapila un A por cada 'b' leída.
- ✓ Si al final de la entrada la pila está vacía (o contiene solo el símbolo inicial) y se ha llegado a un estado de aceptación, la cadena es aceptada (Hopcroft et al., 2002).

El autómata de pila se representa de la siguiente manera:

- ✓ $Q = \{q_0, q_1, q_f\}$
- ✓ $\Sigma = \{a, b\}$
- ✓ $\Gamma = \{A, Z_0\}$
- ✓ q_0 es el estado inicial, q_f es el estado final, y Z_0 es el símbolo inicial de pila (Ortega de la Puente et al., 2006).

Transiciones del autómata (Hopcroft et al., 2002):

- ✓ $(q_0, a, Z_0) \rightarrow (q_0, AZ_0)$: En el estado q_0 , al leer 'a' con Z_0 en el tope de la pila, se apila A.
- ✓ $(q_0, a, A) \rightarrow (q_0, AA)$: En el estado q_0 , al leer 'a' con A en el tope, se apila otra A.
- ✓ $(q_0, b, A) \rightarrow (q_1, \epsilon)$: Al leer 'b' con A en el tope, se desapila A y se transita al estado q_1 .
- ✓ $(q_1, b, A) \rightarrow (q_1, \epsilon)$: En el estado q_1 , al leer 'b' con A en el tope, se desapila A.
- ✓ $(q_1, \epsilon, Z_0) \rightarrow (q_f, Z_0)$: Al final de la entrada, con Z_0 en el tope, se transita al estado final q_f .

4.3.1. Relación con otros autómatas y jerarquía de Chomsky

Los autómatas de pila reconocen los lenguajes libres de contexto (Tipo 2 en la Jerarquía de Chomsky), lo que los sitúa en un punto intermedio entre los autómatas finitos (lenguajes regulares, Tipo 3) y las máquinas de Turing (lenguajes recursivamente enumerables, Tipo 0) (Hopcroft et al., 2002). Esta mayor capacidad expresiva se debe a la memoria de pila que permite contar y anidar estructuras, características esenciales para definir la sintaxis de los lenguajes de programación (Ortega de la Puente et al., 2006).

Utilidad de los autómatas de pila

La principal utilidad de los autómatas de pila en el ámbito de la informática teórica y práctica es su capacidad para modelar y analizar la sintaxis de los lenguajes de programación (Hopcroft et al., 2002). La mayoría de los lenguajes de programación modernos tienen una sintaxis que puede describirse mediante gramáticas libres de contexto, lo que implica que un autómata de pila puede reconocer su estructura. En la práctica, el análisis sintáctico (parsing) se basa en autómatas de pila deterministas (DPDA), que son la base de los analizadores sintácticos (parsers) utilizados en los compiladores (Ortega de la Puente et al., 2006, p. 168).

4.3.2. Relación con AFD y AFND

Los autómatas de pila son más potentes que los autómatas finitos deterministas y no deterministas (Hopcroft et al., 2002). Mientras que los AFD y AFND solo pueden reconocer lenguajes regulares (Tipo 3), los autómatas de pila pueden reconocer lenguajes libres de contexto (Tipo 2), que incluyen estructuras anidadas como paréntesis balanceados o bloques if-else (Ortega de la Puente et al., 2006). Sin embargo, su poder expresivo sigue siendo limitado en comparación con las máquinas de Turing, ya que no pueden resolver problemas que requieren memoria ilimitada o más de una pila (Hopcroft et al., 2002).

4.4. Construcción de analizadores léxicos desde el autómata.

El análisis léxico es la primera fase del proceso de compilación, y su función principal consiste en leer el flujo de caracteres del programa fuente para agruparlos en una secuencia de tokens, que son las unidades mínimas con significado dentro de un lenguaje de programación (Compiladores57.webnode.mx, 2024). Este proceso se fundamenta en la teoría de autómatas finitos, ya que los patrones que definen los tokens se pueden representar mediante lenguajes regulares y, por tanto, ser reconocidos por autómatas finitos (Hopcroft, Motwani & Ullman, 2002).

4.4.1. Enfoques para la implementación de analizadores léxicos:

Existen diversas estrategias para construir un analizador léxico a partir de un autómata. La elección de una u otra depende de la complejidad del lenguaje y de los objetivos de desarrollo. Entre los enfoques más comunes se encuentran:

- ✓ **Autómata implícito:** Se recorre el código fuente carácter por carácter, acumulando un lexema según las reglas del lenguaje. Una vez que se completa un lexema, se clasifica y se genera un token (franleplant, 2020). Este enfoque es adecuado para principiantes, pues establece un paralelo directo con el autómata que reconoce los tokens. Sin embargo, el código puede complicarse fácilmente y perder estructura (franleplant, 2020).
- ✓ **Autómata explícito codificado directamente:** Se define explícitamente un autómata no determinista con una función de transición que especifica el próximo estado y la acción a ejecutar (acumular un lexema, clasificar un token, reportar error, etc.) (franleplant, 2020). Este enfoque es más estructurado que el anterior, pero requiere representar transiciones lambda y casos no deterministas (franleplant, 2020).
- ✓ **Autómata explícito dirigido por tabla:** Es similar al anterior, pero se separan las reglas de transición del motor que las procesa. De esta manera, la arquitectura del programa está mejor organizada y las responsabilidades están bien definidas (franleplant, 2020). Esta separación facilita el mantenimiento y la evolución del analizador léxico (Hopcroft et al., 2002).

- ✓ **Composición de autómatas:** Se especifica una expresión regular para cada tipo de token. Cada expresión regular se convierte en un autómata finito determinista que se ejecuta en paralelo. El lexema correspondiente al token reconocido es la cadena más larga aceptada por alguno de los autómatas antes de que todos caigan en un estado de rechazo (franleplant, 2020). Esta es la estrategia utilizada por generadores de analizadores léxicos como Lex o Flex (franleplant, 2020).

La tabla de símbolos en el análisis léxico

Durante el análisis léxico, se suele utilizar una tabla de símbolos, una estructura de datos que almacena información sobre los identificadores encontrados en el código fuente (Compiladores57.webnode.mx, 2024). Cuando el analizador léxico encuentra un identificador, lo busca en la tabla de símbolos; si no existe, lo inserta y le asigna un identificador único (Compiladores57.webnode.mx, 2024). Esta información será utilizada por las siguientes fases del compilador, como el análisis semántico y la generación de código (Aho, Sethi & Ullman, 1998).

4.4.2. El proceso de tokenización

El analizador léxico ignora los espacios en blanco, tabulaciones y comentarios, ya que no afectan el funcionamiento del programa (Compiladores57.webnode.mx, 2024). Se basa en una gramática léxica, un conjunto de reglas que determinan cómo se agrupan y clasifican las secuencias de caracteres en tokens (Compiladores57.webnode.mx, 2024). Estas reglas se representan mediante expresiones regulares y autómatas finitos (Compiladores57.webnode.mx, 2024). Un token está formado por un nombre (que indica su tipo) y un valor (el lexema concreto) (Compiladores57.webnode.mx, 2024).

4.5. Construcción de analizadores léxicos mediante metacompiladores.

Un metacompilador (también conocido como compilador-compilador o generador de compiladores) es una herramienta de desarrollo de software que se utiliza principalmente en la construcción de compiladores, traductores e intérpretes para otros lenguajes de programación (EPFL Graph Search, s.f.). La entrada de un metacompilador es un programa escrito en un metalenguaje de programación especializado, diseñado específicamente para el propósito de construir compiladores (EPFL Graph Search, s.f.). El lenguaje del compilador producido se denomina lenguaje objeto (EPFL Graph Search, s.f.).

El tipo más común de metacompilador es el generador de analizadores sintácticos (parser generator), que se enfoca únicamente en el análisis sintáctico. Su entrada es un archivo de gramática, generalmente escrito en forma normal de Backus-Naur (BNF) o BNF extendida (EBNF), que define la sintaxis del lenguaje de programación objetivo. La salida es el código fuente de un analizador sintáctico para ese lenguaje (EPFL Graph Search, s.f.). Sin embargo, los metacompiladores más completos pueden generar no solo el analizador sintáctico, sino también el analizador léxico y el generador de código (Devanbu & Frakes, 1996).

4.5.1. Metacompiladores y análisis léxico

En el contexto del análisis léxico, los metacompiladores permiten automatizar la construcción del analizador léxico (lexer) a partir de una especificación de alto nivel. Esta especificación generalmente consiste en un conjunto de expresiones regulares que definen los patrones de los tokens del lenguaje (Devanbu & Frakes, 1996). A partir de esta descripción, el metacompilador genera el código fuente del analizador léxico, que puede ser compilado e integrado en el compilador final.

Una característica clave de los metacompiladores modernos es que permiten definir el lexer de manera modular y extensible (Casey & Hendren, 2011). Herramientas como MetaLexer, por ejemplo, permiten dividir la especificación léxica de un lenguaje en una colección de módulos más pequeños y reutilizables, y controlar el flujo entre ellos mediante el concepto de meta-tokens y meta-lexing (Casey & Hendren, 2011). Esto facilita el manejo de lenguajes con sublenguajes o con diferentes modos de análisis léxico (Casey & Hendren, 2011).

Ejemplos de metacompiladores para análisis léxico

Existen diversas herramientas que implementan el concepto de metacompilador para la generación de analizadores léxicos. Entre las más conocidas se encuentran:

- ✓ **Lex / Flex:** Son herramientas clásicas que generan analizadores léxicos en C a partir de una especificación de expresiones regulares. Aceptan un archivo de especificación (generalmente con extensión .l) que contiene definiciones de patrones y acciones asociadas, y generan código C que implementa un autómata finito (Aho, Sethi & Ullman, 1998).
- ✓ **MetaLexer:** Es un lenguaje de especificación léxica modular que permite definir lexers de manera extensible y reutilizable. Convierte las especificaciones MetaLexer al popular lenguaje JFlex, lo que permite su integración con herramientas de generación de parsers tradicionales (Casey & Hendren, 2011).
- ✓ **Metatool:** Es una herramienta que, a partir de una especificación que incluye expresiones regulares para el lexer y una gramática libre de contexto para el parser, genera el analizador léxico, la estructura de datos del árbol de derivación y el analizador sintáctico (Devanbu & Frakes, 1996).

4.5.2. Ventajas del uso de metacompiladores

El uso de metacompiladores para la construcción de analizadores léxicos ofrece varias ventajas significativas:

- ✓ **Reducción del esfuerzo de desarrollo:** El programador solo necesita especificar los patrones léxicos en un formato de alto nivel, y el metacompilador genera automáticamente el código del analizador (Di Giacomo, 2018).

- ✓ **Mayor confiabilidad:** El código generado automáticamente es menos propenso a errores que el código escrito manualmente.
- ✓ **Mantenibilidad:** Las especificaciones son más fáciles de modificar y extender que el código del analizador escrito a mano.
- ✓ **Modularidad:** Herramientas como MetaLexer permiten organizar la especificación léxica en módulos reutilizables y extensibles (Casey & Hendren, 2011).

Actividad 1: Defina y presente ejemplos de autómatas de pilas.

1. Definición Formal de Autómata de Pila (AP)

Un Autómata de Pila (también conocido como Pushdown Automaton - PDA) es un modelo matemático de computación que extiende el Autómata Finito No Determinístico (AFND) añadiéndole una pila como memoria auxiliar de tipo LIFO (Last-In, First-Out).

Formalmente, un Autómata de Pila se define como una 7-tupla:

$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

Donde cada componente tiene el siguiente significado:

Símbolo	Nombre	Descripción
Q	Conjunto finito de estados	Representa todos los posibles estados en los que puede estar el autómata durante su ejecución.
Σ	Alfabeto de entrada	Conjunto finito de símbolos que pueden ser leídos de la cinta de entrada.
Γ	Alfabeto de pila	Conjunto finito de símbolos que pueden ser almacenados en la pila.
δ	Función de transición	$\delta: Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow P(Q \times \Gamma^*)$ Es el corazón del autómata. Toma el estado actual, un símbolo de entrada (o ϵ = transición épsilon sin consumir entrada) y el símbolo en la cima de la pila, y devuelve un conjunto de posibles nuevos estados y una cadena de símbolos que reemplazarán a la cima de la pila.
$q_0 \in Q$	Estado inicial	El estado en el que el autómata comienza su ejecución.
$Z_0 \in \Gamma$	Símbolo inicial de pila	El símbolo que se encuentra en el fondo de la pila al inicio. Marca el "fondo" de la pila.
$F \subseteq Q$	Conjunto de estados finales o de aceptación	Estados en los que, al terminar de leer la entrada y vaciar (o no) la pila, el autómata acepta la cadena.

Interpretación de la Función de Transición

La función de transición se interpreta de la siguiente manera:

$$\delta(q, a, X) = \{(p_1, \gamma_1), (p_2, \gamma_2), \dots, (p_n, \gamma_n)\}$$

Significado: Si el autómata está en el estado q , lee el símbolo a de la entrada, y el símbolo en la cima de la pila es X , entonces puede:

- Cambiar al estado p_i .
- Reemplazar el símbolo X en la cima de la pila por la cadena γ_i .

Casos especiales:

- Si $\gamma = \epsilon$, se desapila el símbolo X .
- Si $\gamma = X$, la pila no cambia.
- Si $\gamma = YX$, se apila Y y luego X , quedando Y en la cima.

2. Ejemplos de Autómatas de Pila

2.1. Ejemplo 1: Lenguaje de Paréntesis Balanceados ($a^n b^n$)

Descripción del Lenguaje

El lenguaje $L_1 = \{ a^n b^n \mid n \geq 1 \}$ consiste en cadenas con un número igual de 'a's y 'b's, donde todas las 'a's preceden a todas las 'b's.

Ejemplos de cadenas válidas: ab, aabb, aaabbb

Ejemplos de cadenas inválidas: a, abb, aab, aba

Definición Formal del AP

Definimos un Autómata de Pila $P_1 = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$:

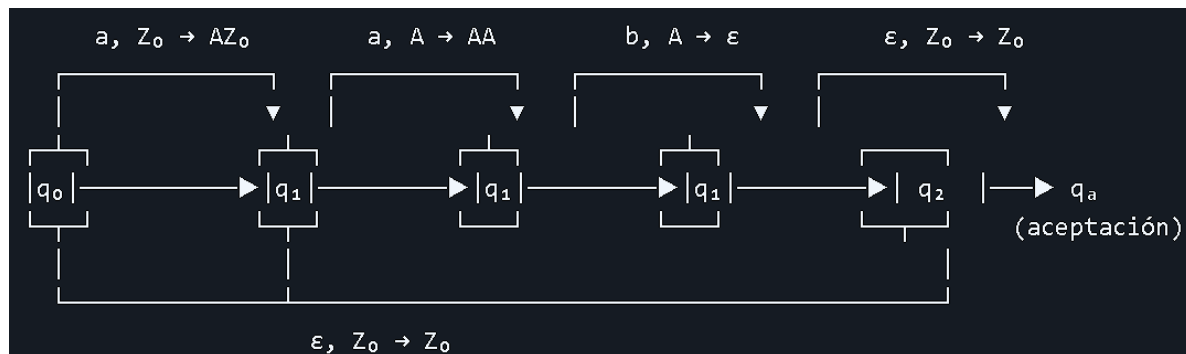
Componente	Valor
Q	$\{q_0, q_1, q_2, q_a\}$
Σ	$\{a, b\}$
Γ	$\{A, Z_0\}$
q_0	q_0
Z_0	Z_0
F	$\{q_a\}$

Función de Transición (δ)

Transición	Interpretación
$\delta(q_0, a, Z_0) = \{(q_1, A Z_0)\}$	Lee la primera 'a', apila 'A' sobre el fondo Z_0 , pasa a q_1 .
$\delta(q_1, a, A) = \{(q_1, A A)\}$	Lee 'a' adicional, apila otra 'A'.
$\delta(q_1, b, A) = \{(q_1, \epsilon)\}$	Lee una 'b', desapila una 'A'.

$\delta(q_1, \epsilon, Z_0) = \{(q_2, Z_0)\}$	Cuando no hay más entrada y solo queda el fondo, pasa a q_2 .
$\delta(q_2, \epsilon, Z_0) = \{(q_a, Z_0)\}$	Transición final al estado de aceptación.

Representación Gráfica



Simulación de la Cadena "aaabbb"

Paso	Estado	Entrada Restante	Pila (cima → fondo)	Transición Aplicada
1	q_0	aaabbb	Z_0	$\delta(q_0, a, Z_0) = (q_1, A Z_0)$
2	q_1	aabbb	$A Z_0$	$\delta(q_1, a, A) = (q_1, A A)$
3	q_1	abbb	$A A Z_0$	$\delta(q_1, a, A) = (q_1, A A A)$
4	q_1	bbb	$A A A Z_0$	$\delta(q_1, b, A) = (q_1, A A Z_0)$
5	q_1	bb	$A A Z_0$	$\delta(q_1, b, A) = (q_1, A Z_0)$
6	q_1	b	$A Z_0$	$\delta(q_1, b, A) = (q_1, Z_0)$
7	q_1	ϵ	Z_0	$\delta(q_1, \epsilon, Z_0) = (q_2, Z_0)$
8	q_2	ϵ	Z_0	$\delta(q_2, \epsilon, Z_0) = (q_a, Z_0)$

Resultado: ¡Cadena ACEPTADA!

2.2. Ejemplo 2: Palíndromos de Longitud Par (ww^R)

Descripción del Lenguaje

El lenguaje $L_2 = \{ w w^R \mid w \in \{0,1\}^* \}$ consiste en cadenas que son palíndromos de longitud par (una cadena seguida de su reverso).

Ejemplos de cadenas válidas: 00, 11, 0110, 101101, 001100

Ejemplos de cadenas inválidas: 01, 10, 010, 1100

Idea del Funcionamiento

1. El autómata lee la primera mitad de la cadena y apila cada símbolo.
2. En el punto medio (que se determina de forma no determinística), cambia de estado.

3. Lee la segunda mitad y desapila un símbolo por cada símbolo leído, verificando que coincidan.
4. Si al final la pila está vacía y se ha consumido toda la entrada, la cadena es aceptada.

Definición del AP

$$P_2 = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

Componente	Valor
Q	$\{q_0, q_1, q_a\}$
Σ	$\{0, 1\}$
Γ	$\{0, 1, Z_0\}$
q_0	q_0
Z_0	Z_0
F	$\{q_a\}$

Transiciones

- Fase de apilado (q_0):
 - $\delta(q_0, 0, Z_0) = \{(q_0, 0, Z_0)\}$ y $\delta(q_0, 0, 0) = \{(q_0, 0, 0)\}$, $\delta(q_0, 0, 1) = \{(q_0, 0, 1)\}$
 - $\delta(q_0, 1, Z_0) = \{(q_0, 1, Z_0)\}$ y $\delta(q_0, 1, 0) = \{(q_0, 1, 0)\}$, $\delta(q_0, 1, 1) = \{(q_0, 1, 1)\}$
- Cambio de fase (no determinístico):**
 - $\delta(q_0, \epsilon, 0) = \{(q_1, 0)\}$ y $\delta(q_0, \epsilon, 1) = \{(q_1, 1)\}$ (supone que ya estamos en la segunda mitad)
- Fase de desapilado y verificación (q_1):**
 - $\delta(q_1, 0, 0) = \{(q_1, \epsilon)\}$ (si lee 0 y la cima es 0, desapila)
 - $\delta(q_1, 1, 1) = \{(q_1, \epsilon)\}$ (si lee 1 y la cima es 1, desapila)
- Aceptación:
 - $\delta(q_1, \epsilon, Z_0) = \{(q_a, Z_0)\}$

Simulación de "0110"

Paso	Estado	Entrada	Pila	Acción
1	q_0	0110	Z_0	Lee 0, apila 0 $\rightarrow$ Pila: 0 Z_0
2	q_0	110	0 Z_0	Lee 1, apila 1 $\rightarrow$ Pila: 1 0 Z_0
3	q_0	10	1 0 Z_0	Transición ϵ: asume que ya está en la segunda mitad
4	q_1	10	1 0 Z_0	Lee 1, coincide con cima (1), desapila $\rightarrow$ Pila: 0 Z_0
5	q_1	0	0 Z_0	Lee 0, coincide con cima (0), desapila $\rightarrow$ Pila: Z_0

6	q_1	ε	Z_0	Transición ε : pasa a q_a
---	-------	---------------	-------	---

Resultado: ¡Cadena ACEPTADA!

2.3. Ejemplo 3: Expresiones Aritméticas con Paréntesis Anidados

Descripción

Consideremos el lenguaje de expresiones aritméticas simples con paréntesis balanceados.

$L_3 = \{ \text{cadenas con paréntesis '(' y ')'} \text{ correctamente balanceados} \}$

Este es un problema práctico que cualquier compilador debe resolver.

AP Simplificado para Paréntesis Balanceados

Componente	Valor
Q	$\{q_0, q_a\}$
Σ	$\{ (,) \}$
Γ	$\{ (, Z_0 \}$
q_0	q_0
Z_0	Z_0
F	$\{q_a\}$

Transiciones

- $\delta(q_0, (, Z_0) = \{(q_0, (Z_0)\} \rightarrow$ Abre paréntesis, lo apila
- $\delta(q_0, (, () = \{(q_0, (()\} \rightarrow$ Abre paréntesis anidado, apila otro
- $\delta(q_0,), () = \{(q_0, \varepsilon)\} \rightarrow$ Cierra paréntesis, desapila uno
- $\delta(q_0, \varepsilon, Z_0) = \{(q_a, Z_0)\} \rightarrow$ Si la pila solo tiene el fondo, acepta

Simulación de "(()())"

1. Lee '(' $\rightarrow$ apila '('
2. Lee '(' $\rightarrow$ apila '('
3. Lee ')' $\rightarrow$ desapila '('
4. Lee '(' $\rightarrow$ apila '('
5. Lee ')' $\rightarrow$ desapila '('
6. Lee ')' $\rightarrow$ desapila '(' (solo queda Z_0)
7. Fin de entrada, transición $\varepsilon \rightarrow$ acepta

3. Relación con AFD y AFND: Poder Computacional

Es fundamental comprender por qué el Autómata de Pila es más poderoso que los Autómatas Finitos.

Característica	AFD / AFND	Autómata de Pila (AP)
Memoria	Finita (estados)	Ilimitada (pila)
Tipo de Memoria	Sin memoria adicional	Pila LIFO
Lenguajes Reconocidos	Lenguajes Regulares (Tipo 3)	Lenguajes Libres de Contexto (Tipo 2)
Capacidad de Contar	No puede contar más allá de un número fijo	Puede contar con la pila (ej: $a^n b^n$)
Estructuras Anidadas	No puede reconocer anidamiento profundo	Puede reconocer anidamiento arbitrario
Ejemplo Clásico	Reconocer tokens (identificadores, números)	Reconocer expresiones con paréntesis balanceados
Complejidad	Lineal ($O(n)$)	Lineal ($O(n)$)

¿Por qué AFD no puede reconocer $a^n b^n$?

Un Autómata Finito tiene un número fijo de estados. Para reconocer $a^n b^n$, necesitaría un estado para cada posible valor de 'n'. Como 'n' puede ser arbitrariamente grande, necesitaría infinitos estados, lo cual no está permitido en un AFD. El Autómata de Pila, en cambio, utiliza la pila para "contar" las 'a's y luego verificar que haya el mismo número de 'b's.

El Autómata de Pila en la Jerarquía de Chomsky

La Jerarquía de Chomsky clasifica los lenguajes formales en cuatro niveles:

Lenguajes Recursivamente Enumerables (Máquinas de Turing) - Tipo 0



Lenguajes Sensibles al Contexto (Autómatas Linealmente Acotados) - Tipo 1



Lenguajes Libres de Contexto (Autómatas de Pila) - Tipo 2 ← ¡Aquí estamos!



Lenguajes Regulares (Autómatas Finitos) - Tipo 3

El Autómata de Pila es el modelo teórico que reconoce exactamente los Lenguajes Libres de Contexto, que son la base de la mayoría de los lenguajes de programación modernos.

4. Utilidades y Aplicaciones del Autómata de Pila

El Autómata de Pila no es solo un concepto teórico; tiene aplicaciones prácticas fundamentales en la informática:

4.1. Construcción de Compiladores (Análisis Sintáctico)

- Los analizadores sintácticos (parsers) como LL(1), LR(0), SLR, LALR y LR(1) están basados en la teoría de Autómatas de Pila.
- Herramientas como YACC (Yet Another Compiler Compiler) y Bison generan analizadores sintácticos a partir de gramáticas libres de contexto, implementando internamente un Autómata de Pila.

4.2. Procesamiento de Lenguajes Naturales

- Los AP se utilizan para analizar la estructura gramatical de oraciones en procesamiento de lenguaje natural.
- Permiten manejar estructuras recursivas como cláusulas subordinadas.

4.3. Validación de Documentos Estructurados

- Validación de XML y HTML: verificar que las etiquetas estén correctamente anidadas y balanceadas.
- Validación de JSON: verificar que los corchetes y llaves estén balanceados.

4.4. Evaluación de Expresiones Aritméticas

- Las calculadoras y los intérpretes de expresiones utilizan pilas para evaluar expresiones con paréntesis anidados y operadores de diferente precedencia (Algoritmo Shunting-yard de Dijkstra).

4.5. Reconocimiento de Patrones Avanzados

- Búsqueda de patrones que requieren "memoria" de lo que se ha visto antes, como en herramientas de análisis de logs y detección de intrusiones.

4.6. Teoría de la Computación

- El AP es un paso fundamental para entender la jerarquía de Chomsky y la capacidad de las máquinas computacionales.
- Sirve como puente conceptual entre los autómatas finitos y las máquinas de Turing.

5. Conclusión de la Actividad 1

En esta actividad se ha abordado el concepto del Autómata de Pila como una extensión natural de los Autómatas Finitos que incorpora una memoria auxiliar en forma de pila. Esta adición, aparentemente simple, confiere a la máquina un poder computacional significativamente mayor, permitiéndole reconocer los Lenguajes Libres de Contexto, que son la base de la sintaxis de casi todos los lenguajes de programación.

Reflexión Final

- Poder Computacional: El Autómata de Pila es más poderoso que los AFD/AFND porque puede "recordar" una cantidad ilimitada de información a través de la pila, lo que le permite contar y manejar estructuras anidadas, como se demostró en los ejemplos de $a^n b^n$ y palíndromos.

- Limitación: Sin embargo, el AP es menos poderoso que una Máquina de Turing, ya que su acceso a la memoria está restringido a la cima de la pila, mientras que una Máquina de Turing puede acceder a cualquier posición de su cinta.

Importancia en la Ingeniería Informática

- Fundamento Teórico: El AP proporciona la base matemática para la construcción de analizadores sintácticos, una de las partes más complejas y críticas de un compilador.
- Aplicación Práctica: Herramientas como Flex (para análisis léxico) y Bison (para análisis sintáctico) son implementaciones prácticas de la teoría de autómatas, y su correcto uso depende del entendimiento profundo de estos conceptos.
- Relevancia en el Tema 4: Entender el Autómata de Pila es esencial para comprender que el análisis léxico (reconocido por AFD) es solo el primer paso. La siguiente fase, el análisis sintáctico, requiere el poder del AP para validar la estructura del programa fuente.

Actividad 2: Lexer para archivos docker mediante expresiones regulares

Se construyó un analizador léxico en Python para verificar archivos Dockerfile, utilizando expresiones regulares para reconocer los tokens del lenguaje. El lexer identifica instrucciones como FROM, RUN, COPY, ENV, EXPOSE, CMD, entre otras, así como identificadores, números, operadores, caracteres especiales y comentarios. Además, detecta errores léxicos cuando encuentra caracteres no reconocidos.

Paso a paso de la Construcción:

1. Definición de tokens: Se estableció una lista de tokens con sus respectivas expresiones regulares, incluyendo palabras clave del Dockerfile, identificadores, números, operadores, caracteres especiales y comentarios. Los tokens se ordenan de más específicos a más generales, y el token ERROR se coloca al final para capturar cualquier carácter no reconocido.

2. Implementación del lexer: Se desarrolló la función lexer() que combina todos los patrones en una sola expresión regular utilizando el módulo (re) de Python. La función recorre el texto buscando coincidencias, clasifica cada token encontrado, ignora espacios en blanco y comentarios, y devuelve los tokens como un generador.

3. Manejo de errores: Se agregó un token ERROR que captura cualquier carácter que no coincida con los patrones definidos, reportando la línea y columna donde ocurre el error. Esto permite detectar caracteres inválidos en el archivo Dockerfile.

4. Función principal: La función main() recibe el nombre del archivo como argumento (o usa un archivo por defecto), lee el contenido y ejecuta el lexer, mostrando los tokens en formato de tabla con su tipo, lexema, línea y columna.

Ejemplos de ejecución:

Ejemplo 1: Dockerfile1 (imagen base simple)

Entrada (pruebas/Dockerfile1):

```
# Dockerfile1 - Imagen base simple
FROM ubuntu:22.04
RUN apt-get update
CMD ["echo", "Hola mundo"]
```

Salida:

Analizando: pruebas/Dockerfile1

TOKEN	LEXEMA	LÍNEA	COLUMNA
FROM	FROM	2	1
IDENTIFIER	ubuntu	2	6
COLON	:	2	12
IDENTIFIER	22.04	2	13
RUN	RUN	3	1
IDENTIFIER	apt-get	3	5
IDENTIFIER	update	3	13
CMD	CMD	4	1
LBRACKET	[	4	5
QUOTE	"	4	6
IDENTIFIER	echo	4	7
QUOTE	"	4	11
COMMA	,	4	12
QUOTE	"	4	14
IDENTIFIER	Hola	4	15
IDENTIFIER	mundo	4	20
QUOTE	"	4	25
RBRACKET	]	4	26

Interpretación: El lexer reconoció correctamente la instrucción FROM con su identificador ubuntu y la versión 22.04, la instrucción RUN con sus argumentos, y la instrucción CMD con el comando entre corchetes, incluyendo las comillas como tokens independientes.

Ejemplo 2: Dockerfile2 (configuración de entorno)

Entrada (pruebas/Dockerfile2):

```
# Dockerfile2 - Configuración de entorno
FROM python:3.12
WORKDIR /app
COPY . /app
ENV PORT=8080
EXPOSE 8080
CMD ["python", "app.py"]
```

Salida:

Analizando: pruebas/Dockerfile2

TOKEN	LEXEMA	LÍNEA	COLUMNA
FROM	FROM	2	1
IDENTIFIER	python	2	6
COLON	:	2	12
IDENTIFIER	3.12	2	13
WORKDIR	WORKDIR	3	1
SLASH	/	3	9
IDENTIFIER	app	3	10
COPY	COPY	4	1
DOT	.	4	6
SLASH	/	4	8
IDENTIFIER	app	4	9
ENV	ENV	5	1
IDENTIFIER	PORT	5	5
EQUALS	=	5	9
IDENTIFIER	8080	5	10
EXPOSE	EXPOSE	6	1
IDENTIFIER	8080	6	8
CMD	CMD	7	1
LBRACKET	[	7	5
QUOTE	"	7	6
IDENTIFIER	python	7	7
QUOTE	"	7	13
COMMA	,	7	14
QUOTE	"	7	16
IDENTIFIER	app.py	7	17
QUOTE	"	7	23
RBRACKET	]	7	24

Interpretación: El lexer reconoció la imagen base python:3.12, el directorio de trabajo /app, la copia del directorio actual al contenedor, las variables de entorno, los puertos expuestos y el comando de ejecución. Los caracteres como :, / y . son reconocidos como tokens específicos (COLON, SLASH, DOT).

Ejemplo 3: Dockerfile3 (configuración avanzada)

Entrada (pruebas/Dockerfile3):

```
# Dockerfile3 - Configuración avanzada
ARG VERSION=1.0
LABEL version="1.0" maintainer="equipo@uneg.edu"
VOLUME /data
USER root
RUN apt-get update && apt-get install -y curl
HEALTHCHECK --interval=30s CMD curl -f http://localhost/
```

Salida:

Analizando: pruebas/Dockerfile3

TOKEN	LEXEMA	LÍNEA	COLUMNA
ARG	ARG	2	1
IDENTIFIER	VERSION	2	5
EQUALS	=	2	12
IDENTIFIER	1.0	2	13
LABEL	LABEL	3	1
IDENTIFIER	version	3	7
EQUALS	=	3	14
QUOTE	"	3	15
IDENTIFIER	1.0	3	16
QUOTE	"	3	19
IDENTIFIER	maintainer	3	21
EQUALS	=	3	31
QUOTE	"	3	32
IDENTIFIER	equipo	3	33
AT	@	3	39
IDENTIFIER	uneg.edu	3	40
QUOTE	"	3	48
VOLUME	VOLUME	4	1
SLASH	/	4	8
IDENTIFIER	data	4	9
USER	USER	5	1
IDENTIFIER	root	5	6
RUN	RUN	6	1

IDENTIFIER	apt-get	6	5
IDENTIFIER	update	6	13
AMPERSAND	&	6	20
AMPERSAND	&	6	21
IDENTIFIER	apt-get	6	23
IDENTIFIER	install	6	31
DASH	-	6	39
IDENTIFIER	y	6	40
IDENTIFIER	curl	6	42
HEALTHCHECK	HEALTHCHECK	7	1
DASH	-	7	13
DASH	-	7	14
IDENTIFIER	interval	7	15
EQUALS	=	7	23
IDENTIFIER	30s	7	24
CMD	CMD	7	28
IDENTIFIER	curl	7	32
DASH	-	7	37
IDENTIFIER	f	7	38
IDENTIFIER	http	7	40
COLON	:	7	44
SLASH	/	7	45
SLASH	/	7	46
IDENTIFIER	localhost/	7	47

Interpretación: El lexer reconoció correctamente la instrucción ARG con su valor 1.0, la instrucción LABEL con múltiples etiquetas incluyendo un correo electrónico (reconociendo @ como el token AT), la instrucción VOLUME, USER, RUN con encadenamiento de comandos usando &&, y la instrucción HEALTHCHECK con sus opciones y el comando final.

Actividad 3: Lexer para L utilizando metacompilador.

Documentación del proceso de creación del lexer

La construcción del analizador léxico para el lenguaje L (subconjunto de Rust) se llevó a cabo siguiendo una metodología incremental, apoyada en la herramienta Flex y el compilador GCC, bajo el sistema operativo Windows con MinGW. A continuación, se detalla el proceso completo, desde la definición del lenguaje hasta la integración del menú interactivo y la organización del repositorio.

Definición del lenguaje objetivo: El primer paso consistió en delimitar el subconjunto de Rust que sería reconocido por el lexer. Se optó por un conjunto representativo que incluyera las categorías léxicas más comunes, pero sin llegar a cubrir todas las características del lenguaje

completo, para mantener la implementación manejable y educativa. La selección se basó en el análisis de programas Rust típicos y en la experiencia previa con el lenguaje.

- **Palabras clave:** se incluyeron 35 palabras reservadas que abarcan estructuras de control (*if, else, loop, while, for*), declaraciones (*fn, let, mut, const, static*), manejo de visibilidad (*pub, use, mod, crate*), conceptos de orientación a objetos (*struct, enum, impl, trait*) y operadores especiales (*match, return, break, continue, async, await, move, ref, dyn, where, in, entre otras*).
- **Tipos primitivos:** se seleccionaron los tipos numéricos con signo y sin signo de 8 a 128 bits (*i8, i16, i32, i64, i128, u8, u16, u32, u64, u128*), los de punto flotante (*f32, f64*), y los tipos *bool, char, str, String, Vec, Option y Result*.
- **Literales:** se definieron patrones para enteros decimales con y sin sufijo de tipo, flotantes, booleanos, caracteres y cadenas con escapes básicos.
- **Operadores y delimitadores:** se incluyeron todos los operadores aritméticos, lógicos, de comparación, de asignación compuesta, de bits, así como los operadores de rango (*.., ...*), flecha (*->, =>*), doble dos puntos (*::*) y los delimitadores estándar: paréntesis, llaves, corchetes, punto y coma, coma, dos puntos y punto.
- **Comentarios:** se soportan los comentarios de línea (*//*) y de bloque (*/* ... */*) en su forma no anidada.
- **Manejo de errores:** se contempla un token de error para cualquier carácter que no encaje en los patrones definidos.

Esta definición se formalizó en una tabla que sirvió como guía para la escritura de las reglas en el archivo *.l*.

Estructuración del archivo de especificación Flex

El archivo *rust_lexer.l* se organizó en las tres secciones estándar de Flex:

- **Sección de definiciones:** se colocaron las opciones *%option noyywrap* y *%option yylineno*, y se incluyó la cabecera *<stdio.h>* y las declaraciones de funciones auxiliares. Se definieron macros para simplificar patrones, aunque en la práctica se optó por escribir los patrones completos para mayor claridad.
- **Sección de reglas:** se listaron todas las reglas en orden de prioridad. Primero, las palabras clave (cada una en su propia línea para evitar problemas de escape y prioridad), luego los tipos primitivos, los literales booleanos, los números con sufijo, los flotantes, los enteros, los caracteres y cadenas, los operadores de varios caracteres (de mayor a menor longitud para que coincidan correctamente), los operadores de un solo carácter y símbolos, y finalmente la regla general para identificadores. Se incluyeron las reglas para comentarios y espacios en blanco, cuyas acciones están vacías, y al final la regla de error para cualquier otro carácter.
- **Sección de código de usuario:** se implementó la función *imprimir_token()* para dar formato de tabla a la salida, la función *procesar_archivo()* que abre un archivo, establece *yyin* y ejecuta *yylex()*, y la función *main()* que, si recibe un argumento, procesa ese archivo; en caso contrario, muestra un menú interactivo que lista los archivos de prueba disponibles en la carpeta *Pruebas/*.

Integración del menú interactivo

Para facilitar las pruebas y demostraciones, se añadió un menú que permite seleccionar entre varios archivos de prueba ubicados en una subcarpeta `Pruebas`. Este menú se implementó usando la biblioteca estándar de C (`dirent.h` para listar directorios, que funciona en entornos Unix y también en Windows con MinGW). El flujo es:

1. Si no se proporciona ningún argumento al ejecutable, se listan todos los archivos regulares dentro de `Pruebas/`.
2. Se muestra un listado numerado y se espera la selección del usuario.
3. Al elegir una opción, se construye la ruta completa y se llama a `procesar_archivo()`.
4. El análisis se repite hasta que el usuario elija salir (opción 0).

Este enfoque hace que el lexer sea autónomo y fácil de usar para evaluar múltiples ejemplos sin necesidad de reescribir comandos.

Pruebas y depuración

Se prepararon tres archivos de prueba (`prueba1`, `prueba2`, `prueba3`) que contienen fragmentos de código Rust con diferentes combinaciones de tokens. Cada prueba se diseñó para verificar aspectos específicos:

Prueba 1: declaración de variables con tipos y asignaciones.

Prueba 2: funciones con parámetros, retorno y operadores aritméticos y de comparación.

Prueba 3: uso de comentarios, literales con sufijos y errores léxicos deliberados (símbolos no reconocidos).

Durante las pruebas iniciales, se detectaron errores de prioridad entre los tipos y los identificadores, así como problemas con la regla de comentarios de bloque, que se corrigieron ajustando el orden de las reglas y simplificando el patrón del comentario de bloque.

Compilación y generación del ejecutable

Una vez depurado el archivo `.l`, se procedió a la compilación:

```
flex rust_lexer.l
gcc lex.yy.c -o analizador_lexico
```

En Windows, se utilizó la terminal de MinGW (o CMD con las variables de entorno configuradas) y se verificó que tanto flex como gcc estuvieran accesibles. El binario resultante se llamó `analizador_lexico.exe` (en Windows) y se ubicó en el directorio raíz del proyecto.

Organización del repositorio

Para cumplir con el requerimiento de entregar un repositorio con código fuente, ejecutable e informe, se estructuró de la siguiente manera:

```
/rust-lexer/
├── rust_lexer.l          # Archivo de especificación Flex
├── lex.yy.c              # (generado) Código C generado por Flex
├── analizador_lexico.exe # Ejecutable compilado (Windows)
├── Pruebas/              # Carpeta con los archivos de prueba
│   ├── prueba1
│   ├── prueba2
│   └── prueba3
└── README.md            # Documentación del proyecto (pasos de instalación y uso)
```

El archivo README.md contiene instrucciones detalladas para la instalación de Flex y MinGW en Windows (con advertencia sobre evitar rutas con espacios), la compilación del lexer y la ejecución del menú interactivo. También se incluyen ejemplos de salida esperada y una breve explicación del lenguaje L.

Lecciones aprendidas

Durante el proceso, se reforzaron conceptos clave como la prioridad de reglas en Flex, la importancia de colocar las palabras clave antes que los identificadores, y el manejo de caracteres especiales en expresiones regulares. La decisión de implementar un menú interactivo no solo mejoró la experiencia de usuario sino que también demostró la flexibilidad de Flex para integrarse con código C adicional. Asimismo, se comprobó que la instalación en Windows requiere cuidado con las rutas, y que el uso de *%option noyywrap* evita problemas de enlace.

La documentación del proceso, junto con el código funcional y los casos de prueba, constituye la entrega completa que permite verificar el correcto funcionamiento del analizador léxico para el lenguaje L.

Actividad 4: FLEX

En el área de seguridad informática, **FLEX** (el generador de analizadores léxicos) se utiliza para procesar y reconocer patrones en grandes volúmenes de texto o tráfico de red. Para que esta herramienta tenga efecto y sea operativa en tu código, debes integrarla con lenguajes de propósito general o de scripting que manejen la lógica principal de tu programa:

1. Lenguajes principales de integración

- ✓ **C y C++:** Son los lenguajes nativos que genera el propio FLEX. En seguridad, se utilizan para construir herramientas de alto rendimiento, que operan a bajo nivel como sistemas

de detección de intrusos (IDS), escáneres de vulnerabilidades o parsers de bajo nivel.

- ✓ **Python:** Es el lenguaje más versátil en ciberseguridad. Aunque FLEX no genere código Python por sí solo, se compila tu analizador de FLEX a lenguaje máquina (como una biblioteca compartida o ejecutable) y controlarlo o alimentarlo mediante scripts en Python utilizando la biblioteca subprocess o interfaces nativas (Ctypes).

2. Áreas prácticas de aplicación en Ciberseguridad

- ✓ **Análisis de Registros (Log Analysis):** Analizar rápidamente millones de líneas en archivos de *logs* de servidores (Apache, Nginx) o de seguridad (Syslog) para detectar anomalías o intentos de ataque (como inyecciones SQL o fuerza bruta). [1, 2]
- ✓ **Análisis de Tráfico de Red:** Crear analizadores personalizados para procesar capturas de paquetes (pcap) o registros de cortafuegos y extraer información específica de protocolos.
- ✓ **Procesamiento de Malware:** Construir herramientas capaces de leer y analizar firmas estáticas o patrones de comportamiento dentro de códigos maliciosos y scripts sospechosos (por ejemplo, analizar scripts ofuscados en JavaScript o VBScript).

3. Herramientas de ciberseguridad que utilizan analizadores similares

- ✓ **Snort:** Es un sistema de detección de intrusos (IDS) de código abierto que utiliza motores de análisis de patrones muy similares en su arquitectura para identificar el tráfico malicioso.
- ✓ **Wireshark:** Emplea analizadores léxicos y sintácticos para diseccionar, desensamblar y comprender docenas de protocolos de red a medida que el tráfico es capturado.

4.1. Reflexión e Indagación del Área

En el ámbito de la seguridad informática, el análisis rápido y preciso de flujos de datos o código es crucial para prevenir y detectar amenazas. Herramientas como FLEX (generadoras de analizadores léxicos basados en Autómatas Finitos Deterministas) son ideales debido a su alta eficiencia en el procesamiento de texto carácter por carácter.

Un área crítica donde se puede aplicar FLEX es en el desarrollo de Sistemas de Detección de Intrusos (IDS) o en el Análisis de Reglas de Firewalls de Aplicación Web (WAF). Específicamente, las reglas de Snort (un IDS de código abierto) constituyen un lenguaje formal declarativo ideal para ser procesado por un lexer generado en FLEX. Las reglas de Snort inspeccionan paquetes de red en busca de firmas maliciosas y requieren un análisis léxico ultraveloz para no generar latencia en la red.

4.2. Lenguaje Seleccionado: Reglas de Snort (Subconjunto de Seguridad)

Para este ejercicio, utilizaremos una estructura simplificada de una regla de detección de firmas de Snort.

Una regla típica de Snort tiene la siguiente forma:

```
alert tcp 192.168.1.1 any -> any 80 (msg:"Intrusión Web Detectada"; content:"malware"; sid:100001;)
```

4.3. Tokenización y Especificación en FLEX

A continuación se presenta el diseño de la tokenización mediante un archivo de especificación de FLEX (`snort\_lexer.l`) para analizar este lenguaje de seguridad informática:

```
%{
#include <stdio.h>
%}

%option noyywrap

%%

/* 1. Acciones de Regla (Keywords de Seguridad) */
"alert"      { printf("TOKEN: ACTION_ALERT, Lexema: %s\n",
yytext); }
"log"        { printf("TOKEN: ACTION_LOG, Lexema: %s\n",
yytext); }
"pass"       { printf("TOKEN: ACTION_PASS, Lexema: %s\n",
yytext); }

/* 2. Protocolos de Red */
"tcp"        { printf("TOKEN: PROTO_TCP, Lexema: %s\n",
yytext); }
"udp"        { printf("TOKEN: PROTO_UDP, Lexema: %s\n",
yytext); }
"icmp"       { printf("TOKEN: PROTO_ICMP, Lexema: %s\n",
yytext); }

/* 3. Direcciones y Variables de Red */
"any"        { printf("TOKEN: NET_ANY, Lexema: %s\n",
yytext); }
"EXTERNAL_NET" { printf("TOKEN: VAR_EXT_NET, Lexema: %s\n",
yytext); }
```

```

"HOME_NET"          { printf("TOKEN: VAR_HOME_NET, Lexema: %s\n",
yytext); }
[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+ { printf("TOKEN: IP_ADDRESS,
Lexema: %s\n", yytext); }[cite: 1]

/* 4. Operadores de Dirección (Flujo de Tráfico) */
"->"                { printf("TOKEN: OP_DIRECTIONAL, Lexema: %s\n",
yytext); }

/* 5. Puertos o Identificadores Numéricos */
[0-9]+              { printf("TOKEN: NUMBER, Lexema: %s\n", yytext);
}

/* 6. Opciones de la Regla (Cuerpo) */
"msg:"              { printf("TOKEN: OPT_MSG, Lexema: %s\n",
yytext); }
"content:"          { printf("TOKEN: OPT_CONTENT, Lexema: %s\n",
yytext); }
"sid:"              { printf("TOKEN: OPT_SID, Lexema: %s\n",
yytext); }

/* 7. Delimitadores y Cadenas de Texto */
"\"\"\"[^\"]*\"\"\" { printf("TOKEN: STRING_LITERAL, Lexema: %s\n",
yytext); }
"("                 { printf("TOKEN: L_PAREN, Lexema: %s\n",
yytext); }
")"                 { printf("TOKEN: R_PAREN, Lexema: %s\n",
yytext); }
";"                 { printf("TOKEN: SEMICOLON, Lexema: %s\n",
yytext); }

/* 8. Ignorados y Errores */
[ \t\n]+           { /* Ignorar espacios, tabulaciones y saltos de
línea */ }[cite: 1]
.                   { printf("ERROR LÉXICO: Carácter no permitido en
reglas de seguridad: %s\n", yytext); }[cite: 1]

%%

int main(int argc, char **argv) {
    if (argc > 1) {
        FILE *file = fopen(argv[1], "r");
        if (!file) {
            perror("Error al abrir el archivo de reglas");
            return 1;
        }
        yyin = file;
    }
}

```

```
    }  
    yylex();  
    return 0;  
}
```

a. Análisis Léxico con FLEX en el Área de Seguridad Informática

4.4.1 Lenguajes Aplicables

En el área de seguridad informática, FLEX es una herramienta fundamental para el análisis léxico de diversos lenguajes especializados. Entre los más destacados se encuentran:

YARA - Lenguaje para Reglas de Identificación de Malware

YARA es una herramienta utilizada para identificar y clasificar muestras de malware mediante la creación de reglas descriptivas. Utiliza FLEX para su análisis léxico. El lexer de YARA tokeniza el texto de las reglas en componentes como identificadores, cadenas, números y operadores.

ClamAV - Firmas de Detección de Virus

ClamAV, un software de escaneo de código abierto, utiliza firmas para detectar malware. Estas firmas incluyen:

- Hashes de archivos
- Expresiones regulares básicas
- Cadenas fijas
- Metafirmas que combinan múltiples condiciones

Wireshark - Filtros de Visualización

Wireshark utiliza FLEX para el análisis léxico de sus filtros de visualización y algunos formatos de archiv . Esto permite procesar expresiones complejas para el filtrado de tráfico de red.

libpcap - Filtros de Captura de Paquetes

La biblioteca libpcap, utilizada por herramientas como tcpdump y Wireshark, implementa el análisis de expresiones de filtro de paquetes (como las de Berkeley Packet Filter - BPF) utilizando FLEX.

Scanners de Seguridad

Scanners como Nmap y Snort utilizan analizadores léxicos basados en FLEX para procesar:

- Reglas de detección de intrusiones
- Scripts de escaneo
- Configuraciones de seguridad
- Expresiones de filtrado

4.4.2. Tokenización para YARA

A continuación se presenta la tokenización del lenguaje YARA basada en la documentación oficial:

Token Type	Descripción	Ejemplo en YARA
Keywords	Palabras clave del lenguaje	rule, private, global, meta, strings, condition
Identifiers	Nombres definidos por el usuario	Nombres de reglas, variables internas
String Identifiers	Identificadores de cadenas (inician con \$)	\$string1, \$hex_string
String Count	Conteo de ocurrencias (inician con #)	#string1
String Offset	Posiciones de cadenas (inician con @)	@string1
Text Strings	Cadenas de texto literal	"This is a string"
Hex Strings	Patrones hexadecimales entre llaves	{ 01 02 03 ?? }
Regular Expressions	Patrones de expresiones regulares	/abc[0-9]+/i
Operators	Operadores matemáticos y lógicos	==, !=, <=, >=, <<, >>
Logical Operators	Conectores lógicos	and, or, not
Constants	Valores booleanos fijos	true, false
Functions	Funciones incorporadas de YARA	matches, contains, startswith, endswith
Numbers	Números decimales o hexadecimales	1024, 0x400
String Modifiers	Modificadores de comportamiento de cadena	wide, ascii, nocase, fullword, xor

4.4.3. Ejemplo de Código FLEX para YARA

A continuación se muestra un ejemplo simplificado de cómo se implementaría un analizador léxico para YARA con FLEX:

```
%{  
#include <stdio.h>  
#include <string.h>
```



```

/* Definiciones de tokens para YARA */
#define TOKEN_RULE      1
#define TOKEN_STRING    2
#define TOKEN_CONDITION 3
#define TOKEN_IDENTIFIER 4
#define TOKEN_KEYWORD   5
#define TOKEN_OPERATOR  6
#define TOKEN_NUMBER    7
#define TOKEN_STRING_LITERAL 8

extern int yylineno;
extern char *yytext;
%}

/* Definiciones de patrones */
DIGIT          [0-9]
HEX_DIGIT      [0-9a-fA-F]
LETTER         [a-zA-Z_]
IDENTIFIER     {LETTER}({LETTER}|{DIGIT})*
STRING_ID      \${IDENTIFIER}
COUNT_ID     #{IDENTIFIER}
OFFSET_ID      @{IDENTIFIER}
HEX_NUMBER     0x{HEX_DIGIT}+
NUMBER         {DIGIT}+
STRING_LITERAL \"[^\"]*\"
HEX_STRING     \{{HEX_DIGIT}{2}({HEX_DIGIT}{2})*\\}
REGEX          /[^\n]+/
COMMENT        \/\/.*
WHITESPACE     [ \t\r]+

%%

/* Palabras clave */
rule           { return TOKEN_RULE; }
private        { return TOKEN_KEYWORD; }
global         { return TOKEN_KEYWORD; }
meta           { return TOKEN_KEYWORD; }
strings        { return TOKEN_KEYWORD; }
condition      { return TOKEN_CONDITION; }
and             { return TOKEN_KEYWORD; }
or             { return TOKEN_KEYWORD; }
not            { return TOKEN_KEYWORD; }
true           { return TOKEN_KEYWORD; }
false          { return TOKEN_KEYWORD; }
wide           { return TOKEN_KEYWORD; }
ascii          { return TOKEN_KEYWORD; }
nocase         { return TOKEN_KEYWORD; }

```

```

fullword          { return TOKEN_KEYWORD; }
xor               { return TOKEN_KEYWORD; }

/* Operadores y símbolos */
"=="             { return TOKEN_OPERATOR; }
"!="             { return TOKEN_OPERATOR; }
"<="            { return TOKEN_OPERATOR; }
">="            { return TOKEN_OPERATOR; }
"<<"           { return TOKEN_OPERATOR; }
">>"           { return TOKEN_OPERATOR; }
"+"             { return TOKEN_OPERATOR; }
"-"             { return TOKEN_OPERATOR; }
"*"             { return TOKEN_OPERATOR; }
"/"             { return TOKEN_OPERATOR; }
{"             { return TOKEN_OPERATOR; }
}"             { return TOKEN_OPERATOR; }
"("             { return TOKEN_OPERATOR; }
")"             { return TOKEN_OPERATOR; }
":"            { return TOKEN_OPERATOR; }
"="            { return TOKEN_OPERATOR; }
";"            { return TOKEN_OPERATOR; }

/* Identificadores */
{IDENTIFIER}     { return TOKEN_IDENTIFIER; }

/* Identificadores de cadenas */
{STRING_ID}      { return TOKEN_STRING; }
{COUNT_ID}     { return TOKEN_STRING; }
{OFFSET_ID}     { return TOKEN_STRING; }

/* Números */
{NUMBER}         { return TOKEN_NUMBER; }
{HEX_NUMBER}    { return TOKEN_NUMBER; }

/* Cadenas literales */
{STRING_LITERAL} { return TOKEN_STRING_LITERAL; }

/* Cadenas hexadecimales */
{HEX_STRING}     { return TOKEN_STRING_LITERAL; }

/* Expresiones regulares */
{REGEX}          { return TOKEN_STRING_LITERAL; }

/* Comentarios */
{COMMENT}        { /* Ignorar comentarios */ }

/* Espacios en blanco */

```

```

{WHITESPACE}                { /* Ignorar espacios */ }

\n                            { yylineno++; /* Contar líneas */ }

.                             { printf("ERROR LÉXICO: Carácter no
reconocido: %s\n", yytext); }

%%

int main(int argc, char **argv) {
    if (argc > 1) {
        FILE *file = fopen(argv[1], "r");
        if (!file) {
            perror("Error al abrir el archivo");
            return 1;
        }
        yyin = file;
    }

    int token;
    while ((token = yylex()) != 0) {
        printf("Token: %d, Lexema: %s\n", token, yytext);
    }

    return 0;
}

```

4.4.4. Aplicaciones Prácticas

Análisis de Archivos Maliciosos

- YARA: Analiza archivos para determinar si coinciden con firmas de malware conocidas
- ClamAV: Escanea archivos en busca de virus utilizando firmas definidas

Análisis de Tráfico de Red

- Wireshark: Filtra paquetes de red en tiempo real mediante expresiones tokenizadas
- tcpdump/libpcap: Procesa expresiones de filtro BPF para capturar paquetes específicos

Herramientas de Detección de Intrusiones

- Snort: Procesa reglas de detección de intrusiones
- Suricata: Analiza tráfico en busca de patrones maliciosos

Verificación de Vulnerabilidades

- FLEX mismo ha sido sujeto de análisis de seguridad, como lo demuestra la vulnerabilidad CVE-2006-0459

4.5. Ventajas de Usar FLEX en Seguridad Informática

Rendimiento: FLEX genera código C optimizado que procesa texto extremadamente rápido, crucial para aplicaciones de seguridad en tiempo real.

Precisión: Al basarse en Autómatas Finitos Deterministas (AFD), el tiempo de procesamiento es lineal respecto al tamaño del texto de entrada, ofreciendo predictibilidad contra ataques de denegación de servicio lógicos (ReDoS).

Portabilidad: El código generado es C estándar multiplataforma, facilitando su ejecución en sistemas embebidos (como routers o firewalls de hardware) y sistemas operativos Linux/Windows/Unix.

Modularidad e Integración: FLEX se integra fácilmente con herramientas de parsing como Bison/Yacc para construir analizadores sintácticos completos

Flexibilidad: Permite a los ingenieros de seguridad definir lenguajes específicos de dominio (DSL) adaptados de forma exacta a nuevas firmas de malware o protocolos propietarios de red.

Conclusión.

En conclusión, el desarrollo del presente trabajo ha permitido consolidar los fundamentos teóricos y prácticos del análisis léxico, una fase esencial en el proceso de compilación. A lo largo de las distintas secciones, se ha transitado desde los conceptos abstractos de autómatas y expresiones regulares hasta su materialización en analizadores léxicos funcionales, demostrando la relevancia de esta disciplina en la ingeniería informática.

En el bloque teórico, se abordaron los autómatas finitos y su estrecha relación con las expresiones regulares, estableciendo el Teorema de Kleene como el puente conceptual que permite describir lenguajes regulares desde dos perspectivas complementarias: la generativa y la reconocedora. Asimismo, se exploró la construcción de autómatas a partir de gramáticas regulares, evidenciando la equivalencia entre ambos formalismos y su utilidad en el análisis léxico. Por otro lado, los autómatas de pila se presentaron como un modelo de computación con mayor poder expresivo que los autómatas finitos, capaces de reconocer lenguajes libres de contexto y sentando las bases para el análisis sintáctico en compiladores.

La primera actividad práctica permitió definir formalmente los autómatas de pila y ejemplificar su funcionamiento mediante lenguajes como $a^n b^n$, palíndromos de longitud par y expresiones con paréntesis balanceados. Se evidenció que los autómatas de pila superan a los autómatas finitos en capacidad de contar y manejar estructuras anidadas, lo que los convierte en la herramienta teórica fundamental para el análisis sintáctico. Asimismo, se comprendió que, aunque los autómatas de pila son más poderosos que los AFD y AFND, su poder sigue siendo limitado en comparación con las máquinas de Turing, lo que permite ubicarlos correctamente dentro de la Jerarquía de Chomsky.

La segunda actividad, centrada en la construcción de un analizador léxico para archivos Dockerfile en Python, demostró la efectividad de las expresiones regulares para el reconocimiento de patrones léxicos. El manejo de errores implementado evidenció la importancia de detectar caracteres no reconocidos para garantizar la validez de los archivos procesados. La evolución del lexer, desde una versión inicial que reportaba caracteres válidos como errores hasta la versión final que los reconoce correctamente, reflejó la importancia de un análisis cuidadoso del lenguaje objetivo.

La tercera actividad, consistente en el desarrollo de un lexer para un subconjunto de Rust utilizando Flex como metacompilador, mostró cómo las herramientas de metacompilación facilitan la generación de analizadores léxicos eficientes, reduciendo el esfuerzo de desarrollo y mejorando la confiabilidad del código. La experiencia de definir el lenguaje L, estructurar el archivo de especificación Flex y depurar las reglas de prioridad permitió comprender las ventajas y limitaciones de este enfoque.

Por último, la exploración del uso de FLEX en el ámbito de la seguridad informática reveló que los analizadores léxicos trascienden el contexto de los compiladores tradicionales. Herramientas críticas como YARA, Snort y Wireshark dependen de analizadores léxicos para

procesar reglas de detección de malware, filtros de tráfico de red y firmas de intrusiones, demostrando que los fundamentos del análisis léxico tienen aplicaciones directas en áreas de alta demanda profesional.

En conjunto, este trabajo ha permitido comprender que el análisis léxico no es un fin en sí mismo, sino el primer eslabón de una cadena de procesamiento que culmina en la generación de código ejecutable. La correcta definición de los tokens, el diseño cuidadoso de las expresiones regulares y la elección adecuada entre implementación manual o mediante metacompiladores son decisiones que impactan directamente en la eficiencia, confiabilidad y mantenibilidad de los sistemas de procesamiento de lenguajes.

Referencias.

Aho, A. V., Sethi, R., & Ullman, J. D. (1998). *Compiladores: Principios, técnicas y herramientas*. Pearson Educación.

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2002). *Introducción a la teoría de autómatas, lenguajes y computación* (2ª ed.). Pearson Educación.

Analizador léxico. (s.f.). En Wikipedia. Recuperado el 12 de julio de 2026, de https://es.wikipedia.org/wiki/Analizador_l%C3%A9xico

Compiladores1.webnode.mx. (2024, 23 de noviembre). *Análisis léxico - Compiladores*. Recuperado de <https://compiladores1.webnode.mx/ice-dea/>

Linz, P. (2011). *An Introduction to Formal Languages and Automata* (5th ed.). Jones & Bartlett Learning.

Ortega de la Puente, A., Alfonseca Moreno, M., de la Cruz Echeandía, M., & Pulido, E. (2006). *Compiladores e intérpretes: Teoría y práctica*. Pearson Educación.

Compiladores57.webnode.mx. (2024). *Análisis léxico*. Recuperado el 12 de julio de 2026, de <https://compiladores57.webnode.mx/analisis-lexico/>

franleplant. (2020). *Una clase sobre Lexers*. GitHub. Recuperado de https://github.com/franleplant/lex_lecture

Casey, A., & Hendren, L. (2011). *MetaLexer: A Modular Lexical Specification Language*. ACM.

Devanbu, P., & Frakes, W. (1996). *Metatool: A Meta-Compiler for Language Engineering* [Documento de clase]. University of California, Davis. Recuperado de <https://www.cs.ucdavis.edu/~devanbu/teaching/260/metnotes.pdf>

Di Giacomo, F. (2018). *Metacasanova: A High-performance Meta-compiler for Domain-specific Languages* (Tesis doctoral). Università Ca' Foscari Venezia.

EPFL Graph Search. (s.f.). *Compiler-compiler*. Recuperado de <https://graphsearch.epfl.ch/en/concept/70097>

Jurado Málaga, E. (2008). *Teoría de autómatas y lenguajes formales*. Universidad de Extremadura, Servicio de Publicaciones.